

Aim:

You are designing a grid layout for a next-generation server farm in Hyderabad (India's IT hub) or a cellular network tower distribution system across a metropolitan area.

- The grid is modelled as an $N \times N$ matrix.
- You must place N high-frequency antennas on this grid.
- To avoid electromagnetic interference, no two antennas can share the same row, column, or diagonal line of sight.

This is essentially the N-Queens Problem, a classic in computer science and combinatorial optimization.

Input Format:

- A single integer N , representing the size of the grid ($N \times N$) and number of antennas to place.

Output Format:

- If a solution exists, it should print in the following format:

```
Successful Placement
.A..
...A
A...
..A.
```

where, **A** → antenna placed in the grid and **.** → empty cell.

- If no solution exists:

```
No solution exists
```

- If input is invalid ($N \leq 0$ or $N > MAX$):

```
Invalid
```

Note: Partial code is provided. Complete the missing logic in **isSafe()**, **solveNQueens()**, and **printSolution()**.

Source Code:

```
antennaNetworking.c
```

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 20

// declare grid to place antennas
int board[MAX][MAX];

bool isSafe(int row, int col, int N) {
    for(int j=0;j<col;j++){
        if(board[row][j]==1)
            return false;
    }
}
```

```

        return false;
    }
    for(int i=row+1,j=col-1;i<N && j>=0;i++,j--){
        if(board[i][j]==1)
            return false;
    }
    return true;
// implement safety check
}

bool solveNQueens(int col, int N) {
    if(col>=N)
        return true;

    for(int row=0;row<N;row++){
        if(isSafe(row,col,N)){
            board[row][col]=1;
            if(solveNQueens(col +1, N))
                return true;
            board[row][col]=0;
        }
    }
    return false;
// backtracking solution to place antennas column by column
}

void printSolution(int N) {
    for(int i=0;i<N;i++){
        for(int j=0;j<N;j++){
            if(board[i][j]==1)
                printf("A");
            else
                printf(".");
        }
        printf("\n");
    }
// display final board
}

int main() {
    int N;
    printf("antennas(N): ");
    scanf("%d", &N);
    if (N <= 0 || N > MAX) {
        printf("Invalid\n");
        return 0;
    }
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;
    if (solveNQueens(0, N)) {
        printf("Successful Placement\n");
        printSolution(N);
    }
}

```

```

    } else {
        printf("No solution exists");
    }
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
antennas(N):
4
Successful Placement
..A.
A...
...A
.A..

Test Case - 2
User Output
antennas(N):
2
No solution exists

Test Case - 3
User Output
antennas(N):
-2
Invalid